

XHAND SDK 接口说明文档

版本记录	3
X-hand API 参考	3
XHandControl	3
枚举设备	3
打开设备	3
获取所有手 ID	5
关闭设备	6
读取设备信息	6
读取版本信息	7
发送控制指令	8
读取状态	8
关节标定（旧版标定，之后逐步废弃）	9
关节标定（模具）	10
传感器复位	12
读取手类型	13
读取手序列号	14
读取手名字	15
设置手名字	15
数据结构	16
结构体	16
枚举	21
通信 ID	22
手指关节 ID	22
灵巧手 ID	22

版本记录

X-hand API 参考

XHandControl

XHandControl 是灵巧手 sdk 主要 API 接口类，与灵巧手的通信均需要通过调用该类的相应方法来实现

枚举设备

接口功能

枚举设备，可选择单独一个设备进行通信

接口声明

C++

```
std::vector<std::string> enumerate_devices(const std::string& comm_type)
```

注意事项

无

参数说明

参数名称	类型	是否必选	示例值	说明
comm_type	const std::string &	是	"RS485" 或 "EtherCAT"	枚举当前可用的设备名

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	std::vector<std::string>		返回串口设备名或者网卡名

打开设备

打开串口设备

接口功能

根据指定的端口和波特率尝试连接 X-hand 设备

接口声明

```
C++
ErrorStruct open_serial(const std::string& port, uint32_t baudrate)
```

注意事项

连接成功后才能与 X-hand 进行正常通信

当前串口使用的波特率为 3000000

参数说明

该接口的参数说明如下：

参数名称	类型	是否必选	示例值	说明
port	const std::string &	是	“com3”	
baudrate	uint32_t	是	3000000	通信串口使用的波特率

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	ErrorStruct	无	error_code 为 0 代表成功，否则代表失败，参考 错误结构体

打开 EtherCAT 设备

接口功能

打开 EtherCAT 设备

接口声明

```
C++
```

```
ErrorStruct open_ethercat(const std::string& ifname)
```

注意事项

连接成功后才能与 X-hand 进行正常通信

参数说明

该接口的参数说明如下：

参数名称	类型	是否必选	示例值	说明
ifname	const std::string &	是	enps0	

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	ErrorStruct	无	error_code 为 0 代表成功，否则代表失败，参考 错误结构体

获取所有手 ID

接口功能

获取所有已连接的手的 ID

接口声明

```
C++
std::vector<uint8_t> list_hands_id()
```

注意事项

必须先调用打开设备接口成功后，调用该接口才能返回连接的手 ID

参数说明

无

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	<code>std::vector<uint8_t></code>		所有已连接的手 ID

关闭设备

接口功能

关闭当前打开的 X-hand 设备

接口声明

```
C++
void close_device()
```

注意事项

退出程序前需关闭 X-hand 设备，保证设备正常退出

参数说明

无

返回值

无

读取设备信息

接口功能

读取打开的 X-hand 设备信息

接口声明

```
C++
std::pair<ErrorStruct, DeviceInfo_t> read_device_info()
```

注意事项

如果未读取到信息，所有信息数据都为 0

参数说明

无

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	<code>std::pair<ErrorStruct, DeviceInfo_t></code>		参考 设备信息 ， 错误结构体 ，如果 ErrorStruct 的 error_code 不为 0，则读取失败

读取版本信息

接口功能

读取 X-hand 设备的版本信息

接口声明

C++

```
std::pair<ErrorStruct, std::string> read_version(const uint8_t device_id, const uint8_t component_id)
```

注意事项

如果未读取到信息，返回空字符串

参数说明

参数名称	类型	是否必选	示例值	说明
component_id	<code>const uint8_t</code>	是	0	详见 ID 说明
device_id	<code>const uint8_t</code>	是	0	整手 ID

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	<code>std::pair<ErrorStruct, std::string></code>		参考 错误结构体 ，如果 ErrorStruct 的 error_code 不为 0，则读取失败

发送控制指令

接口功能

下发关节指令

接口声明

```
C++
ErrorStruct send_command(const uint8_t device_id,const HandCommand_t&
command)
```

注意事项

res 为预留参数，无需赋值

参数说明

该接口的参数说明如下：

参数名称	类型	是否必选	示例值	说明
device_id	const uint8_t	是	0	整手 ID
command	const HandCommand_t &	是	HandCommand_t 结构体	HandCommand_t 结构体

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	std::pair<ErrorStruct, DeviceInfo_t>		参考 错误结构体 ，如果 ErrorStruct 的 error_code 不为 0，则 f 发送失败

读取状态

接口功能

读取当前 x-hand 状态信息

接口声明

```
C++
std::pair<ErrorStruct, HandState_t> read_state( uint8_t device_id, bool
force_update = false)
```

注意事项

如果为串口通信，下发控制指令后会自动刷新 HandState_t，如果需要强制刷新 HandState_t，需要一定耗时

参数说明

参数名称	类型	是否必选	示例值	说明
device_id	const uint8_t	是	0	整手 ID
force_update	bool	否	false	如果为 true,会发送状态刷新指令，会有一定阻塞时间

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	std::pair<ErrorStruct, HandState_t>		返回值参考 HandState_t 结构体 错误结构体

关节标定（旧版标定，之后逐步废弃）

接口功能

标定 X-hand 关节控制参数

接口声明

```
C++
ErrorStruct calibrate_joint(const uint8_t device_id, const uint8_t step,const
```

```
std::vector<int8_t> angles_limit = {})
```

注意事项

标定 X-hand 所有关节的控制参数，标定参数异常可能导致关节运动异常，请谨慎使用

参数说明

该接口的参数说明如下：

参数名称	类型	是否必选	示例值	说明
device_id	const uint8_t	是	0	整手 ID
step	uint8_t	是	[1,5]	标定分为 5 个阶段，需要按照顺序执行
angles_limit	const std::vector<int8_t>	否	{ 0, 90, -70, 80, 0, 90, 12, -8, 0, 110, 0, 110, 0, 110, 0, 110, 0, 110, 0, 110, 0, 110};	关节运动的限位角度，不同型号可能有不同值，详细信息，请咨询技术人员

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	ErrorStruct	无	error_code 为 0 代表成功，否则代表失败，参考 错误结构体

关节标定（模具）

接口功能

借助模具自动标定 X-hand 关节控制参数

接口声明

C++

```
ErrorStruct calibrate_joint_by_mold(const uint8_t device_id, const std::string& tools, const uint8_t step,
```

```
    const std::vector<int8_t> calibrated_angles = {}, const  
    std::vector<int8_t> angles_limit= {})
```

注意事项

标定 X-hand 所有关节的控制参数，标定参数异常可能导致关节运动异常，请谨慎使用；

标定过程中尽量保证 X-hand 紧贴模具

参数说明

该接口的参数说明如下：

参数名称	类型	是否必选	取值	说明
device_id	const uint8_t	是	0	整手 ID
tools	const std::string &	是	"S1", "D1"	标定模具型号
step	uint8_t	是	Int, 取值为 1 或 2	单模具只需要 1 步，多模具由模具具体个数决定
calibrated_angles	const std::vector<int8_t>	否	- 如果 tools 取 S1, 则只需一步, 则 step=1; calibrated_angles = [10, 60, 30, 0, 10, 10, 10, 10, 10, 10, 10] - 如果 tools 取 "D1", 则分两步, step=1, calibrated_angles=[10, 30, 60, 0, 10, 10, 10, 10, 10, 10, 10]; step=2,	标定角度，根据模具的固定角度进行确定

			calibrated_angles=[50, 60, 45, 10, 30, 30, 30, 30, 30, 30, 30, 30]	
angles_limit	,const std::vector<int8_t>	否	M2.25: { 0, 105, -60, 90, -10, 105, -10, 10, 0, 110, 0, 110, 0, 110, 0, 110, 0, 110, 0, 110, 0, 110};	关节运动的限位角度，不同型号可能有不同值，详细信息，请咨询技术人员

调用方式

- 单模具标定：调用一次 `calibrate_joint_by_mold`，step 值为 1，传递一次 `calibrated_angles`
- 多模具标定：需要调用两次 `calibrate_joint_by_mold`，step 值分别为 1 和 2，传递两次 `calibrated_angles`，具体值如上表所示

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	ErrorStruct	无	error_code 为 0 代表成功，否则代表失败，参考 错误结构体

传感器复位

接口功能

复位指尖压力传感器

接口声明

```
C++
ErrorStruct reset_sensor(const uint8_t device_id,const uint8_t sensor_id)
```

注意事项

部分使用情况下，需要先复位传感器，保证传感器数值的准确性

参数说明

参数名称	类型	是否必选	示例值	说明
sensor_id	const uint8_t	是	[17,21]	Sensor id 从 17 开始，从大拇指依次增加

返回值

该接口的返回值说明如下：

参数名称	类型	是否必选	示例值	说明
device_id	const uint8_t	是		整手 ID
返回值	ErrorStruct	是		error_code 为 0 代表成功，否则代表失败，参考 错误结构体

读取手类型

接口功能

读取手的类型(L/R)

接口声明

C++

```
std::pair<ErrorStruct, std::string> get_hand_type(uint8_t device_id)
```

注意事项

无

参数说明

参数名称	类型	是否必选	示例值	说明
device_id	const uint8_t	是	0	整手 ID

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	std::pair<ErrorStruct, std::string>		返回值参考 错误结构体

读取手序列号

接口功能

读取手的序列号

接口声明

C++

```
std::pair<ErrorStruct, std::string> get_serial_number(uint8_t device_id)
```

注意事项

无

参数说明

参数名称	类型	是否必选	示例值	说明
device_id	const uint8_t	是	0	整手 ID

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	std::pair<ErrorStruct, std::string>		返回值参考 错误结构体

读取手名字

接口功能

读取手的名字

接口声明

C++

```
std::pair<ErrorStruct, std::string> get_hand_name(uint8_t device_id)
```

注意事项

无

参数说明

参数名称	类型	是否必选	示例值	说明
device_id	const uint8_t	是	0	整手 ID

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	std::pair<ErrorStruct, std::string>		返回值参考 错误结构体

设置手名字

接口功能

设置手的名字

接口声明

C++

```
ErrorStruct set_hand_name(const uint8_t device_id,const std::string& name);
```

注意事项

名字不支持中文，最长不超过 30 个字符

参数说明

参数名称	类型	是否必选	示例值	说明
device_id	const uint8_t	是	0	整手 ID
name	const std::string&	是	"test_hand"	

返回值

该接口的返回值说明如下：

参数名称	类型	示例值	说明
返回值	ErrorStruct		返回值参考 错误结构体

数据结构

该章节介绍 X-hand API 涉及的结构体和枚举。

结构体

HandCommand_t

本结构体记录 **HandCommand_t** 信息。结构体声明如下所示。

C++

```
typedef struct {
    uint16_t id;
    int16_t kp;    // 比例增益参数
    int16_t ki;    // 积分系数
    int16_t kd;    // 微分增益参数
    float position; // 设定位置
    uint16_t tor_max; // 力矩上限
    uint16_t mode; // 模式
    uint16_t res0;
    uint16_t res1;
    uint16_t res2;
    uint16_t res3;
} __attribute__((packed)) FingerCommand_t;

typedef struct {
```



```
std::array<FingerCommand_t, 12> finger_command;
} __attribute__((packed)) HandCommand_t;
```

该结构体的参数说明如下：

参数名称	类型	是否必选	示例值	说明

HandState_t

本结构体记录 **HandState_t** 信息。结构体声明如下所示。

```
C++
typedef struct {
    int8_t fx;
    int8_t fy;
    uint8_t fz;
} __attribute__((packed)) PXSR_ForceData;

typedef struct {
    PXSR_ForceData calc_force;
    std::array<PXSR_ForceData, 120> raw_force;
    std::array<uint8_t, 20> temperature;
    uint8_t calc_temperature;
} __attribute__((packed)) SensorData_t;

typedef struct {
    uint64_t id;
    float position;
    uint64_t torque;
    uint64_t default0;
    uint64_t default1;
    uint64_t default2;
    uint64_t default3;
    uint64_t default4;
    uint64_t default5;
    uint64_t default6;
    uint64_t default7;
} __attribute__((packed)) FingerState_t;
```

```
typedef struct {
    std::array<FingerState_t, 12> finger_state;
    std::array<SenserData_t, 5> senser_data;
} __attribute__((packed)) HandState_t;
```

该结构体的参数说明如下：

参数名称	类型	是否必选	示例值	说明

DeviceInfo_t

本结构体记录 **DeviceInfo_t** 相关信息。结构体声明如下所示。

```
C++
typedef struct {
    std::array<int16_t, 12> position_stretching{};
    std::array<int16_t, 12> position_closing{};
    std::array<int16_t, 12> position_zero{};
    std::array<int16_t, 12> position_init{};
    std::array<int8_t, 12> angle_stretching{};
    std::array<int8_t, 12> angle_closing{};
} __attribute__((packed)) HandParam_t;

// 新校准参数
typedef struct {
    std::array<char, 11> tools; // [S1, D1]
    uint8_t step; // [1,2]
    std::array<int8_t, 12> calibrated_angles; // 标定角度（单位：度）：从 spec 获取
    std::array<int8_t, 12> lower_angles; // 上位机获取的下限角度：从 spec 获取
    std::array<int8_t, 12> upper_angles; // 上位机获取的上限角度：从 spec
```

获取

```
} __attribute__((packed)) CalibrationParams;

typedef union {
    HandParam_t handparam; CalibrationParams calibr_params;
} CalibrationUnion;

typedef struct {
    uint16_t resverse;
    char ev_hand;    //左手 'L'/'l' 还是右手 'R'/'r'
    uint8_t hand_id; //灵巧手 ID
    std::array<char, 32> serial_number; //序列号, 16 位
    std::array<char, 32> name;          //灵巧手名称
    HandParam_t hand_param;            //校准参数
    uint8_t iap_flag;                  //升级标志
    uint8_t is_calibrated;
} __attribute__((packed)) DeviceInfo_t;
```

该结构体的参数说明如下：

参数名称	类型	是否必选	示例值	说明

CalibrationErrorInfo

本结构体记录 **CalibrationErrorInfo** 相关信息。结构体声明如下所示。

C++

```
typedef struct {
    CalibrationStatus calib_status;    // 校准状态信息, uint8_t
    uint16_t joint_number;            // 错误时返回错误的关节 number (0~11) 或者 0xffff (不对应某一关节参数时)
    // 只有 CALIBRATION_ERROR_ADC_OUT_OF_RANGE /
```

```
CALIBRATION_ERROR_ADC_PRE_UPPER_ERROR /
CALIBRATION_ERROR_ADC_PRE_LOWER_ERROR 这三个错误会携带参数，
其它错误时参数默认为 0.;
uint16_t calibrated_angle;           // 错误值：校准角度
uint16_t calibrated_adc_value;       // 错误值：校准原始值
uint16_t intercept;                  // 错误值：校准插值
} __attribute__((packed)) CalibrationErrorInfo;
```

该结构体的参数说明如下：

参数名称	类型	是否必选	示例值	说明

ErrorStruct

本结构体记录 **ErrorStruct** 相关信息。结构体声明如下所示。

```
C++
tstruct ErrorStruct {
    ErrorStruct({});
    ErrorStruct(int code, std::string message) {
        error_code = code;
        error_message = message;
    }
    int error_code{0};
    std::string error_message;
    void reset() {
        error_code = 0;
        error_message = "";
    }
    // 重载 bool 运算符
```

```
explicit operator bool() const { return error_code == 0; }  
};
```

枚举

关节索引

手指关节索引值枚举

```
C++  
enum FingerIndex{  
    THUMB_BEND=0,  
    THUMB_ROTA_1=1,  
    THUMB_ROTA_2=2,  
    INDEX_BEND=3,  
    INDEX_JOINT_1=4,  
    INDEX_JOINT_2=5,  
    MIDDLE_JOINT_1=6,  
    MIDDLE_JOINT_2=7,  
    RING_JOINT_1=8,  
    RING_JOINT_2=9,  
    PINKY_JOINT_1=10,  
    PINKY_JOINT_2=11,  
}
```

标定错误索引

自动标定错误信息索引值枚举

```
C++  
typedef enum : uint8_t {  
    CALIBRATION_SUCCESS = 0,           // 操作成功  
    CALIBRATION_ERROR_ANGLE_OUT_OF_RANGE,    // 角度超出范围  
    CALIBRATION_ERROR_ADC_OUT_OF_RANGE,      // ADC 超出范围  
    CALIBRATION_ERROR_ADC_PRE_UPPER_ERROR,   // ADC 预测上边界参数错误  
    CALIBRATION_ERROR_ADC_PRE_LOWER_ERROR,   // ADC 预测下边界参数错误  
}
```

```
CALIBRATION_SAVE_PARAM_ERROR,           // 校准时保存参数错误
CALIBRATION_INPUT_PARAM_ERROR,          // 校准输入参数错误
} CalibrationStatus;
```

通信 ID

手指关节 ID

灵巧手由一个掌板、12 个指关节板、5 个指尖传感器板组成。每一个板由唯一 ID 作为通信的唯一标识，且不重复。

其中指关节板和指尖传感器板的 ID，不可更改，其值如下：

名称	ID	名称	ID	名称	ID	名称	ID	名称	ID
大拇指偏摆	0	食指偏摆	3	中指 1 关节	6	无名指 1 关节	8	小指 1 关节	10
大拇指 1 关节	1	食指 1 关节	4	中指 2 关节	7	无名指 2 关节	9	小指 2 关节	11
大拇指 2 关节	2	食指 2 关节	5						
大拇指传感器	0x11	食指传感器	0x12	中指传感器	0x13	无名指传感器	0x14	小指传感器	0x15

灵巧手 ID

灵巧手 ID 默认为 0，用户可以将修改。范围 0~125。

掌板 ID 是灵巧手 ID 的最高位置 1，这样可以和指关节 ID 和指尖传感器 ID 区分开

例如：灵巧手 ID 设置为 3（0x03），实际通讯使用 0x83 作为掌板 ID